

ok, gracias ya tengo la gantt corregida a como la presentaste en lo sugerido por ti. Ahora vamos con lo siguiente, por cada tarea que tenemos en la gantt, revisa las tablas de las bases de datos aportadas en los archivos de models.py que te subí, y generame una mini-especificación funcional para que sea el esqueleto del programa (vista, template, etc) que se necesitara para lograr los scripts que trabajen en python, como esto es muy extenso para procesar las 79 tareas de una vez, puedes entregarme por grupos y por partes, según te acomode para que el procesamiento no sea tan intenso, dame un esquema que podamos seguir y lo vamos sacando por partes, ¿te parece?

Ha pensado durante 2m 11s

Sí, me parece la forma correcta de continuar. Procesar las 79 tareas en una sola entrega produciría especificaciones demasiado extensas y difíciles de revisar. Conviene trabajar por bloques pequeños, manteniendo siempre la relación entre la Carta Gantt, los modelos existentes y los archivos Django que habrá que construir.

Los modelos ya entregan una base de dominio considerable. models2-administracion.py cubre expresamente banco y conciliación, facturación/cobranza/factoring y rendiciones; además, deja fuera de esta fase la contabilidad general y los asientos automáticos, por lo que evitaremos ampliar accidentalmente el alcance.

1. Método de trabajo

Antes de escribir las mini-especificaciones, prepararé una **matriz de cobertura** para las 79 tareas:

Estado	Significado
Cubierta	Los modelos actuales soportan la función
Parcial	Existe parte del modelo, pero falta alguna entidad o regla
Sin modelo adicional	La tarea corresponde principalmente a vistas, servicios, reportes o templates
Requiere nuevo modelo	La base de datos actual no representa la función
Requiere corrección	El modelo existe, pero debe ajustarse antes de construir la función

Esto permitirá detectar con anticipación qué tareas pueden desarrollarse directamente y cuáles necesitan una migración adicional.

2. Formato de cada mini-especificación

Cada tarea de la Gantt tendrá una ficha uniforme.

Encabezado

- Código y nombre de la tarea.
- Módulo.
- Objetivo funcional.
- Dependencias de la Gantt.
- Estado de cobertura del modelo.

Base de datos

- Modelos utilizados.
- Tablas involucradas.
- Relaciones relevantes.
- Campos obligatorios.
- Estados y transiciones.
- Validaciones existentes.
- Campos o modelos faltantes.
- Migraciones necesarias.

Backend Django

- Vista recomendada: función, View, ListView, CreateView, UpdateView, etc.
- forms.py.
- Servicio de dominio.
- Selectores o consultas.
- Uso de transaction.atomic().
- Uso de select_for_update(), cuando exista movimiento de dinero.
- Señales o tareas programadas, solo cuando correspondan.
- Permisos.
- Auditoría.

- Manejo de errores.

Frontend

- URL.
- Template.
- Componentes de pantalla.
- Campos y botones.
- Filtros.
- Tablas.
- Modales.
- Mensajes de validación.
- Confirmaciones antes de acciones irreversibles.
- Comportamiento JavaScript necesario.

Pruebas y aceptación

- Pruebas unitarias.
- Pruebas de permisos.
- Pruebas de integración.
- Casos normales.
- Casos límite.
- Criterios de aceptación.
- Resultado esperado.

Archivos esperados

Por ejemplo:

administracion/

├─ urls.py

├─ views/

| └─ bancos.py

└─ forms/

```
|   └─ bancos.py
|── services/
|   └─ conciliacion.py
|── selectors/
|   └─ movimientos.py
|── templates/administracion/bancos/
|   ├── banco_list.html
|   ├── banco_form.html
|   └─ banco_confirm_delete.html
└─ tests/
    ├── test_banco_views.py
    ├── test_banco_services.py
    └─ test_banco_permissions.py
```

No asumiré que toda la lógica debe quedar en views.py. Las operaciones financieras y cambios de estado importantes deberían quedar en servicios de dominio reutilizables.

3. División propuesta de las 79 tareas

Parte 1 — Fundamentos transversales

7 tareas

- A001 Levantamiento y alcance.
- A002 Modelo de datos y migraciones.
- A003 Estados y reglas de negocio.
- A004 Permisos y roles.
- A005 Auditoría e historial.
- A006 Gestión de adjuntos.
- A007 UX y navegación.

Esta parte establecerá las convenciones que utilizaremos en todos los módulos.

El modelo abstracto ModeloAuditoria ya registra creación, modificación y usuarios responsables, pero todavía habrá que distinguir entre auditoría básica de registro y eventos de negocio importantes.

Parte 2 — Banco: maestros e importación

5 tareas

- B001 Catálogo de bancos.
- B002 Cuentas bancarias.
- B003 Importación de cartolas.
- B004 Plantillas de mapeo bancario.
- B005 Detección de duplicados.

Los modelos Banco, CuentaBancaria, ImportacionCartola y MovimientoBancario proporcionan una base adecuada. Las restricciones por SHA-256 y fingerprint ayudarán a controlar duplicados.

Preliminarmente, **B004 parece requerir un modelo adicional** para almacenar configuraciones de columnas por banco o formato de cartola.

Parte 3 — Banco: consulta y aplicación inicial

4 tareas

- B006 Lista de movimientos.
- B007 Detalle de movimiento.
- B008 Clasificación manual.
- B009 Aplicaciones bancarias múltiples.

AplicacionMovimientoBancario es el puente central del sistema: distribuye un movimiento hacia pagos de clientes, factoring, entregas de fondo, liquidaciones y devoluciones.

Su validación exige exactamente un destino y evita que la suma aplicada supere el movimiento bancario.

Parte 4 — Facturación y pagos principales

7 tareas

- F001 Clientes y contactos.
- F002 Registro o importación de facturas.
- F003 Lista de facturas.
- F004 Estado de cobranza.
- F005 Registro de pagos.
- F006 Pago aplicado a varias facturas.
- F007 Pagos parciales y múltiples.

No se creará otro modelo de cliente. FacturaVenta y PagoCliente utilizan extintores.Cliente, que ya contiene nombre, RUT, correo, dirección, comuna, teléfono y contacto.

AplicacionPagoFactura ya controla que:

- la factura pertenezca al mismo cliente;
 - las aplicaciones no superen el pago;
 - los pagos aplicados no superen el total de la factura.
-

Parte 5 — Facturación, cobranza y salidas

7 tareas

- F008 Excedentes y pagos sin aplicar.
- F009 Vinculación con cartola.
- F010 Gestión de cobranza.
- F011 Alertas de vencimiento.
- F012 Antigüedad de saldos.
- F013 Reportes y exportación.
- F014 Pruebas del módulo.

Esta parte utilizará principalmente propiedades calculadas, consultas agregadas, filtros, servicios de cobranza y reportes. No todas estas tareas necesitan tablas nuevas.

Parte 6 — Rendiciones: maestros y captura

9 tareas

- R003 Responsables.
- R004 Categorías y subcategorías.
- R002 Crear y editar rendición.
- R001 Lista y búsqueda.
- R005 Entregas de fondo.
- R006 Detalle de gastos.
- R007 Adjuntos y previsualización.
- R008 Validación documental.
- R009 Cálculos.

El modelo actual ya incluye:

- responsable;
- categorías;
- subcategorías;
- rendición;
- entregas de fondo;
- detalle del gasto.

DetalleRendicion contempla estado de revisión, monto aprobado, observaciones, justificación para gastos sin documento y comprobante. También impide aprobar un monto superior al gasto y obliga a justificar documentos faltantes.

Posible ajuste de modelo

Actualmente cada detalle tiene un único campo comprobante. Para permitir varios archivos, fotos o documentos por gasto, probablemente será necesario agregar:

```
class AdjuntoDetalleRendicion(ModeloAuditoria):
```

```
    detalle = models.ForeignKey(
        DetalleRendicion,
        on_delete=models.CASCADE,
        related_name="adjuntos",
    )
```

```
    archivo = models.FileField(...)
```

```
    nombre_original = models.CharField(...)
```

```
    tipo_mime = models.CharField(...)
```

Este punto se resolverá formalmente en R007.

Parte 7 — Rendiciones: workflow y liquidación

8 tareas

- R010 Presentación.
- R011 Revisión y observaciones.
- R012 Aprobación o rechazo.
- R013 Historial de aprobaciones.
- R014 Liquidación.
- R015 Devolución del responsable.
- R016 Reembolso al responsable.
- R017 Conciliación de liquidaciones.

AprobacionRendicion ya representa presentación, revisión, observación, aprobación, rechazo y reapertura.

LiquidacionRendicion valida los tres resultados posibles:

- rendición cuadrada;
- responsable debe devolver;
- empresa debe reembolsar.

También existe control para impedir que los pagos de regularización superen la diferencia de la liquidación.

Parte 8 — Rendiciones: salidas y pruebas

5 tareas

- R018 Alertas.
- R019 Exportación PDF y Excel.
- R020 Dashboard.
- R021 Pruebas.
- R022 Manual.

R018 posiblemente requerirá ampliar AlertaCobranza o crear una alerta transversal, porque la alerta actual está principalmente orientada a facturas, pagos y factoring.

Parte 9 — Conciliación avanzada

8 tareas

- B010 Un movimiento paga varias facturas.
- B011 Conciliación parcial.
- B012 Reglas automáticas.
- B013 Cruce manual asistido.
- B014 Cruce con rendiciones y facturas.
- B015 Exclusiones.
- B016 Reversas y reapertura.
- B017 Cierre del período.

Aquí se concentrará la lógica más sensible del sistema.

Las operaciones deberán implementarse mediante servicios transaccionales, no directamente en templates o formularios:

@transaction.atomic

```
def aplicar_movimiento(...):
```

```
movimiento = (  
    MovimientoBancario.objects  
        .select_for_update()  
        .get(pk=movimiento_id)  
)  
...
```

Preliminarmente, B012 necesitará una representación de reglas automáticas y B016 podría necesitar un modelo de evento o reversa que deje evidencia inmutable.

Parte 10 — Banco: dashboard, alertas y pruebas

4 tareas

- B018 Dashboard.
 - B019 Alertas.
 - B020 Reportes y exportaciones.
 - B021 Pruebas del módulo.
-

Parte 11 — Factoring operativo

7 tareas

- X001 Empresas de factoring.
- X002 Operación de factoring.
- X003 Estados y eventos.
- X004 Anticipo y comisión.
- X005 Confirmación de pago.
- X006 Alertas de mora.
- X007 Cobro con recurso.

Los modelos existentes separan correctamente:

- operación;

- evento;
- flujo monetario.

EventoFactoring conserva el historial operacional y FlujoFactoring representa anticipos, liquidaciones, comisiones, retenciones y cobros con recurso.

Parte 12 — Factoring excepcional y cierre

6 tareas

- X008 Detectar pago directo de factura cedida.
- X009 Devolución al cliente.
- X010 Conciliar devolución.
- X011 Conciliar liquidación del factor.
- X012 Reportes.
- X013 Pruebas.

Esta parte utilizará conjuntamente:

- FacturaVenta;
 - OperacionFactoring;
 - PagoCliente;
 - AplicacionPagoFactura;
 - DevolucionPagoCliente;
 - MovimientoBancario;
 - AplicacionMovimientoBancario.
-

Parte 13 — Calidad global

2 tareas

- Q001 Pruebas end-to-end.
- Q002 Aceptación y ajustes.

Aquí construiremos escenarios completos, no pruebas aisladas:

Cartola

- movimiento bancario
- pago de cliente
- aplicación a facturas
- actualización de cobranza
- conciliación

y:

Entrega de fondo

- rendición
- revisión
- aprobación
- liquidación
- devolución o reembolso
- conciliación bancaria

4. Entregable de cada parte

Cada entrega contendrá:

1. Matriz tarea-modelo.
2. Mini-especificación de cada tarea.
3. Modelos y migraciones faltantes.
4. Estructura de carpetas.
5. URLs.
6. Vistas.
7. Formularios.
8. Servicios.
9. Selectores o consultas.
10. Templates.
11. Permisos.

12. Reglas transaccionales.
13. Pruebas.
14. Criterios de aceptación.
15. Orden recomendado de implementación.

No produciré todavía el código completo de cada módulo. Primero construiremos el **esqueleto funcional y técnico**; después ese documento se podrá convertir en código tarea por tarea sin improvisar arquitectura.

Primer bloque

La siguiente entrega debe comenzar con:

Parte 1 — A001 a A007: fundamentos transversales

Es la base necesaria para fijar la estructura Django, estados, permisos, auditoría, adjuntos y convenciones de navegación que reutilizarán las otras 72 tareas.